

ECE3073 Project Technical Report

Team 103

Prepared by: Sam Neumann and Wenxing Du

Date prepared: May 24th, 2024

1 Introduction

For this project we were tasked with implementing a NIOS-II Real Time Operating System (RTOS) on a DE-10 FPGA board with the functionality to store and display a low-resolution grayscale image. The image must be displayable with the full screen or downscaled into four smaller images.

In addition, this RTOS also provides real time 3x3 blurring, Sobel edge detection, and image flipping processing that can be selected with a basic interface.

The additional design requirements are detailed in table 1.

Table 1: Design Requirements

Category:	Required:	Optimal:
Image resolution:	160 x 120 pixels	
Time for downscaling and displaying:	<250 ms	
Time for all processing operations:	<500 ms	
Time for Edge detection processing:	<350 ms	<150 ms

2 System Components and Design

This section will present an overview of the major software and hardware components of this project.

2.1 NIOS II System and RTOS

The hardware processor used for this project was a full NIOS-II processor that has Programmed Input/Output (PIO) connections to the FPGA interface components such as LEDs, switches, and 7-segment displays, along with PIO connections to other hardware components including the SDRAM memory, Pixel Buffer, and the Convolution Accelerator. All this hardware is synthesised on the FPGA board.

The processor is running a μ C/OS-II RTOS which allows individual tasks to access all the other hardware resources through the PIOs.

The SDRAM memory is used as the storage location for the base image and all the processed images, which can be read and written to by the processor.

2.2 VGA Controller and Pixel buffer

The Nios Processor writes the output image to the Pixel buffer, and then a hardware block called the VGA Controller reads from the Pixel Buffer and writes the contents to the FPGA's VGA port. This dual access Pixel Buffer allows the processor and the VGA port to interface with each other even though they need different clock speeds.

2.3 Convolution Accelerator

This is a hardware system that performs all the operations required for running the 3x3 convolutions used by the blurring and Sobol edge filtering algorithms at much faster speeds than performing all the operations through NIOS-II instructions. The processor software writes the values to the PIO values and reads them from another PIO without doing any of the operations itself. See Appendix D for more details.

2.4 User Interface

Users can choose to display one full 160x120 pixel image or four images reduced to 80x60 pixels on the screen. Keys are used to switch between single and quad-image display modes, as well as trigger functions such as blurring, edge detection and image flipping. Switches are used to select specific image processing operations, such as enabling or disabling blurring and edge detection. The seven-segment display is used to display the processing time and CPU usage of image processing operations, providing users with real-time performance feedback.

2.5 Image Downscaling

The downscale function is used to reduce the original 160x120 pixel image into four small 80x60 pixel images and display them in the four quadrants of the screen. By sampling and averaging the original image, every four pixels are synthesised into a target pixel. The optimised downscale function reduces the function call overhead and utilises the parallel processing capability of FPGA to increase the processing speed.

3 Development and Testing Methodology

This section will discuss the development and testing of the design to ensure it aligns with the requirements. See Appendix C for a full list of technologies used during this development process.

3.1 RTOS Development Process

Throughout the development process of the μ C/OS-II RTOS, we focused on creating logical tasks with appropriate priorities that maximise the performance of the processor, such as the display tasks having a lower priority than the processing tasks because displaying the wrong image that isn't processed is not useful. Another example is that the benchmarking task has the lowest priority as it doesn't need to operate in real time based on the requirements, and having a delay in this task would not be as disruptive to the user as having a delay with any of the other tasks.

Another essential aspect of effective RTOS design is to consider tasks avoiding corruption or deadlock when accessing resources. To avoid corruption we used mutexes to protect all shared resources such as PIOs and Global variables by giving exclusive access to a single task, and to avoid deadlocks we made sure the mutexes followed a strict order when accessing.

Finally, we made sure that all User facing input changes such as switches and buttons were handled with a short Interrupt Service Routine (ISR) that uses a semaphore to signal their service. This makes the input servicing quicker while ensuring it only runs when necessary, unlike polling.

3.2 Benchmarking

Throughout this project we used a task to benchmark the speed of the important algorithms, along with the total utilisation of the processor, to ensure the design matches the timing requirements. This was done with a task that reads the time before and after executing an algorithm, and displaying the difference, which gives us an accurate measure of the time the algorithms take.

3.3 Improvements

The initial implementation of the design did not match the required timing constraints regarding edge detection and processing tasks, therefore multiple improvements were developed and tested. The measurements recorded at each step can be seen in Section 4.

3.3.1 Combined Blurring and Edge Detection

In the original design, blur processing and edge detection were performed separately, which resulted in long combined processing times. To optimise performance, we tried to combine these two image processing operations. During the implementation process, we defined a function that contains a blur processing convolution kernel and an edge detection convolution kernel. When traversing the image pixels, these two convolution kernels are applied simultaneously to calculate the blurring results and edge detection results. Then, store the results of blurring to one target address and the results of edge detection to another target address. In this way, we complete two processing operations in one traversal. This improved the speed of the combined processes negatively impacted the individual edge detection algorithm timing criteria, therefore this change wasn't used.

3.3.2 Reducing Unnecessary Function Calls

At the start of the development process read and write operations to the SDRAM were encapsulated in functions to make the development easier, however we were concerned that this was affecting the efficiency compared to just reading/writing without a function. A function call requires saving return values and states to the stack each it's called and returned, which takes more time.

To test this we created a test of both read and write operations with and without functions (see Appendix B for details) and found a significant difference of 10-30% speed improvement when not using functions.

After seeing these results we removed the use of functions from all instances of SDRAM read/write operations. This improved the speed but still didn't reach the required timing constraints.

3.3.3 Hardware Acceleration for Convolutions

The final improvement we tested was to have our convolution processing be accelerated by hardware designed specifically for the 3x3 convolutions used by the blurring and edge detection algorithms. In addition, this method also significantly reduces the number of memory accesses for each convolution. See Appendix D for more details on how the improvement works.

This method had a massive improvement in speed over implementing the convolutions with normal processor instructions and more memory accesses. Once this improvement was implemented our design exceeded the optimal timing requirements.

4 Results

Once we completed the design process our project met all the functionality requirements and was able to display either one large image or four downscaled images, with the user selected processing options (see Appendix A). Table 2 shows how the timings changed over the development process, and that with the two successful improvements we have achieved the required timing constraints.

Table 2: Timing changes with Improvements

	Flipping	Blurring	Edge Detection	All processing
Before Improvements	53 ms	563 ms	700 ms	1320 ms
With combined blurring and edge detection	53 ms	798 ms	798 ms	798 ms
Without unnecessary functions	35 ms	497 ms	631 ms	1165 ms
Without unnecessary functions and with hardware acceleration	36ms	84 ms	103 ms	223 ms

5 Discussion

The software improvement of combining blur and edge detection functions presents an interesting tradeoff between one criteria and another. If in the future hardware improvements are impossible then this might be worth the downsides, however in this circumstance where hardware changes are much better we didn't consider utilising it.

The software improvement of removing functions is another tradeoff, this time between execution speed without functions and ease of modification in the future. A potential solution in the future would be to test the speed and ease of using C macros (IBM, 2024).

Using hardware acceleration is very effective but also very inflexible because it only works for this particular process of convolutions. In addition, the current design can only manage two convolutions, and in the future refactoring the Verilog HDL to allow for more or larger convolutions would help with more complex tasks. This use of FPGA parallelism for convolutions may also be useful for newer developments in Machine Learning and Convolutional Neural Networks.

6 Conclusion

The implementation we developed contains all the functionality and exceeds the timing constraints by a significant margin, therefore we believe this design is suitable and ready for deployment.

References

IBM. (2024, May 7). Function-like macros. IBM Documentation; IBM.
<https://www.ibm.com/docs/en/i/7.5?topic=directive-function-like-macros>

Appendices

Appendix A: Processed Image Examples

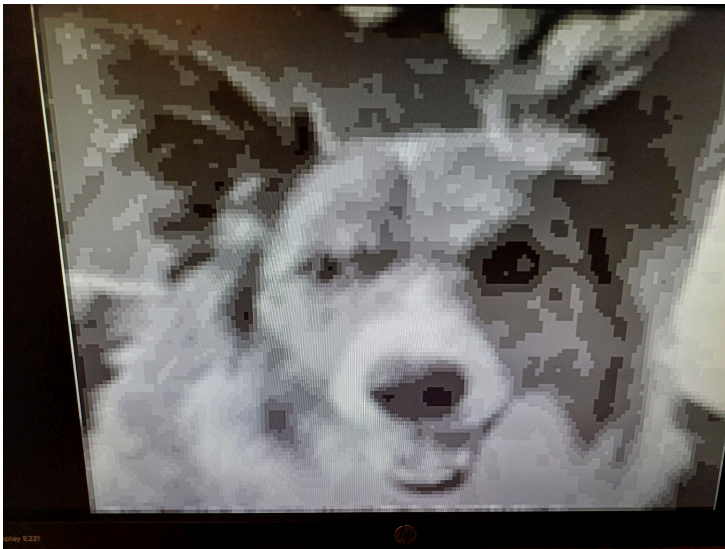
The following images show the functionality of the design.



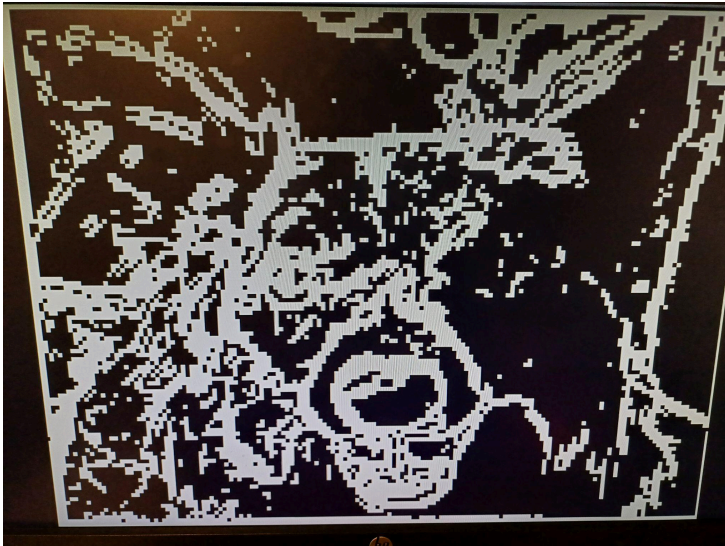
A single image displayed on the full 160 x 120 pixels



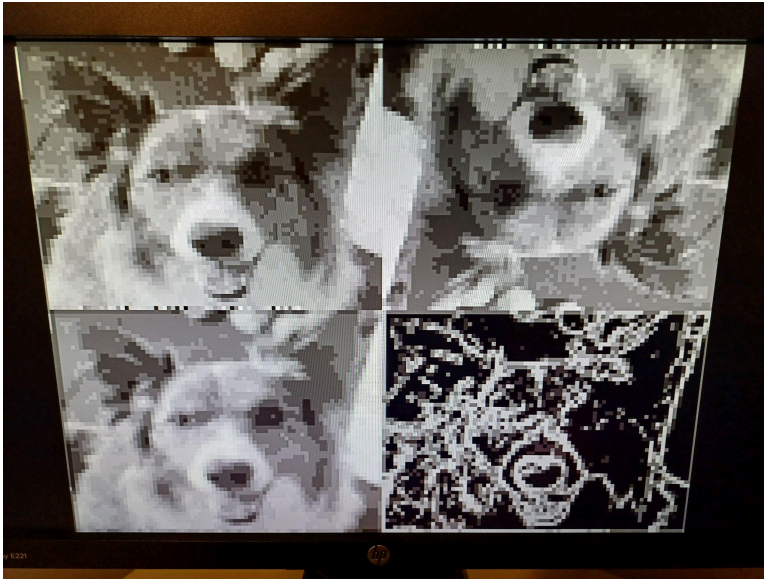
A flipped image displayed on the full 160 x 120 pixels



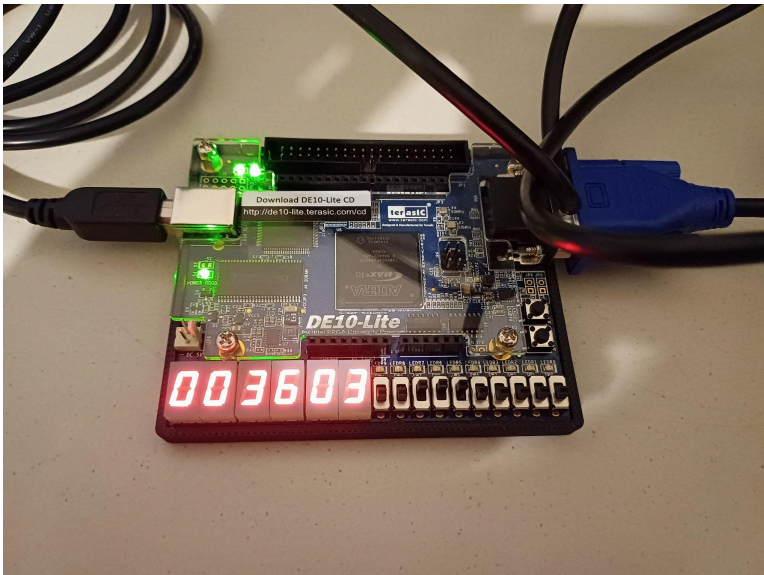
A blurred image image displayed on the full 160 x 120 pixels



An image with edge detection displayed on the full 160 x 120 pixels



Four versions of the image displayed at once



The FPGA board displaying the CPU usage and benchmarking times

Appendix B: Software Improvement Details

Testing Method

To test whether using functions was faster or slower for SDRAM read/write operations, we created a test that checks both read and writing, with and without functions 100,000 times, and ran this test 23 times.

The following code was added the benchmarking task:

```
int testVal;  
int testStartTime;
```

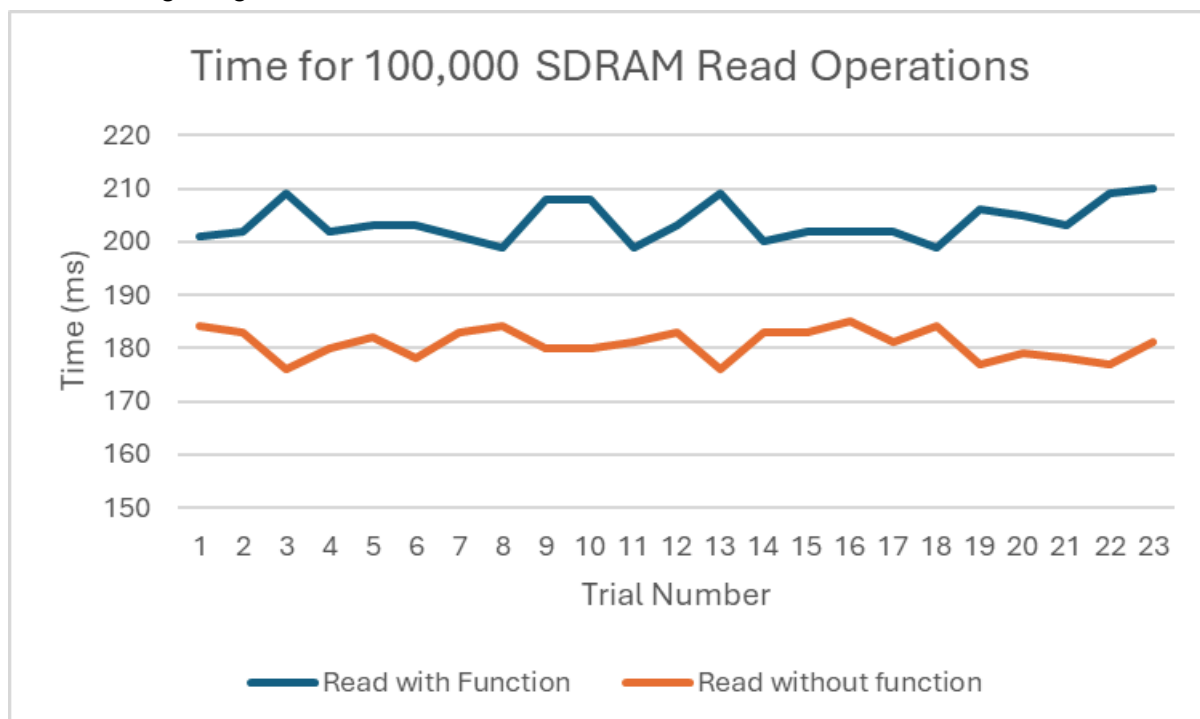
```

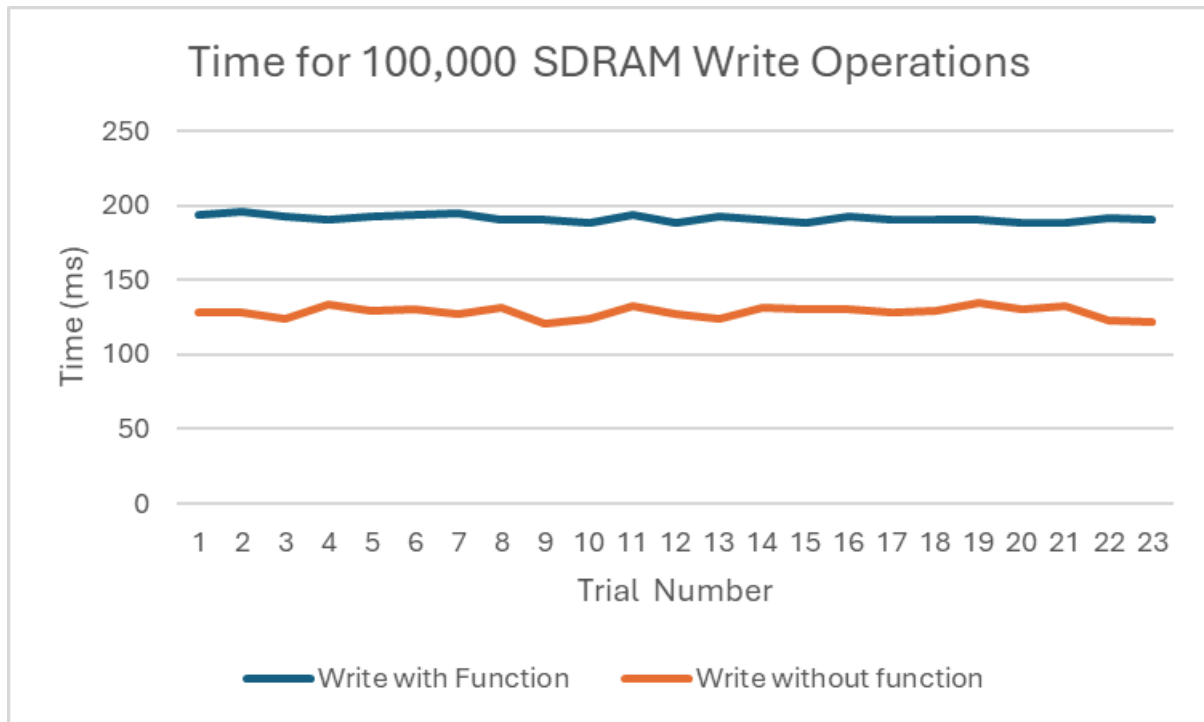
int totalReadTimeFunc = 0, totalWriteTimeFunc = 0;
int totalReadTime = 0, totalWriteTime = 0;
// Test the time it takes to access SDRAM memory
for (int i = 0; i < 100000; i++) {
    testStartTime = OSTimeGet();
    sdramPixelWrite(DEFAULT_SDRAM_BASE_ADDRESS, 0);
    totalWriteTimeFunc += OSTimeGet() - testStartTime;
    testStartTime = OSTimeGet();
    testVal = sdramPixelRead(DEFAULT_SDRAM_BASE_ADDRESS);
    totalReadTimeFunc += OSTimeGet() - testStartTime;
    testStartTime = OSTimeGet();
    IOWR(4*DEFAULT_SDRAM_BASE_ADDRESS, 0, 0 << 24);
    totalWriteTime += OSTimeGet() - testStartTime;
    testStartTime = OSTimeGet();
    testVal = IORD(4*DEFAULT_SDRAM_BASE_ADDRESS, 0) >> 24;
    totalReadTime += OSTimeGet() - testStartTime;
}
printf("Time to read 100000 times with function: %d\n", totalReadTimeFunc);
printf("Time to write 100000 times with function: %d\n",
totalWriteTimeFunc);
printf("Time to read 100000 times without function: %d\n", totalReadTime);
printf("Time to write 100000 times without function: %d\n", totalWriteTime);

```

Results

The following images show the results:





On average reading without functions was 11.2% faster than with functions, and writing was 33.0% faster without function than with functions.

Appendix C: Technologies Used

Hardware Technologies

- DE-10 Lite FPGA (version 10M50DAF484C7G)
- Various monitors with VGA ports for testing

Software Technologies

- DE-10 Lite FPGA Control Panel
- Quartus Prime Lite Edition
- Nios II Software build tools for Eclipse

Appendix D: Explanation of Convolution Hardware Acceleration

This hardware acceleration improves the speed in two separate ways.

Firstly, the edge detection algorithm normally requires two separate convolutions of nine values to be multiplied with the pixel values and then summed, which takes multiple instructions for a single pixel convolution. With the hardware implementation we only need to write and read values from the Convolution Accelerator Hardware through PIOs without doing most of the calculations.

Secondly, the original convolution reads the nine pixels from the SDRAM and then saves the resulting value, which is ten read/write operations. However the Hardware Accelerator stores 6 of the values in registers from the last convolution and only needs to read three values from the SDRAM per pixel and write one value (see the image below) which also improves the performance.

Last convolution

